



4/16/2026

SPOTIFY TRACKS DATA

Music isn't one-dimensional. Why do we force it to be?

Decoding Music DNA: From Misleading Accuracy to Multi-Label Genre Affinity

• Introduction:

- Goal : Predict broad genre families using Spotify audio DNA while testing whether those families align with measurable audio structure

```
import matplotlib.pyplot as plt
import kagglehub
import os
import pandas as pd
import numpy as np
import shap
import pandas as pd
import seaborn as sns
import plotly.graph_objects as go

from sklearn.manifold import TSNE
from sklearn.manifold import TSNE
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.metrics import multilabel_confusion_matrix, classification_report
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import f1_score
from sklearn.metrics import precision_recall_curve, auc
from sklearn.utils import class_weight

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.models import Model
from tensorflow.keras.utils import plot_model
from tensorflow.keras import Input
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
```

• Dataset and genre mapping:

- 114,000 tracks
- 12 raw audio features
- mapping 114 sub-genres to 8 families

```
# 2. Expanded Mapping (Template - fill in the rest of the 114)
genre_mapping = {
    # 1. Acoustic/Folk (Natural/Organic Instruments)
    'acoustic': 'Acoustic/Folk', 'folk': 'Acoustic/Folk', 'bluesgrass': 'Acoustic/Folk',
    'world-music': 'Acoustic/Folk', 'singer-songwriter': 'Acoustic/Folk', 'french': 'Acoustic/Folk',
    'gospel': 'Acoustic/Folk', 'newyork': 'Acoustic/Folk', 'indian': 'Acoustic/Folk',
    'irish': 'Acoustic/Folk', 'mpb': 'Acoustic/Folk', 'reggae': 'Acoustic/Folk',
    'samba': 'Acoustic/Folk', 'sertanejo': 'Acoustic/Folk', 'singerpop': 'Acoustic/Folk',
    'tango': 'Acoustic/Folk', 'turkish': 'Acoustic/Folk', 'breil': 'Acoustic/Folk',
    'country': 'Acoustic/Folk',

    # 2. Blues/Jazz (Groove & Soul based)
    'blues': 'Blues/Jazz', 'jazz': 'Blues/Jazz', 'soul': 'Blues/Jazz', 'r-n-b': 'Blues/Jazz',
    'funk': 'Blues/Jazz', 'gospel': 'Blues/Jazz', 'groove': 'Blues/Jazz', 'reggae': 'Blues/Jazz',

    # 3. Classical (High Dynamic Range / Orchestral)
    'classical': 'Classical', 'opera': 'Classical', 'piano': 'Classical',

    # 4. Electronic (Synthesized/Drum Machine)
    'ambient': 'Electronic', 'breakbeat': 'Electronic', 'chill': 'Electronic', 'club': 'Electronic',
    'deep-house': 'Electronic', 'disco': 'Electronic', 'drum-and-bass': 'Electronic', 'dub': 'Electronic',
    'dubstep': 'Electronic', 'electro': 'Electronic', 'electronic': 'Electronic', 'garage': 'Electronic',
    'hardstyle': 'Electronic', 'house': 'Electronic', 'jazz': 'Electronic', 'industrial': 'Electronic',
    'minimal-techno': 'Electronic', 'new-age': 'Electronic', 'progressive-house': 'Electronic',
    'sleep': 'Electronic', 'study': 'Electronic', 'trance': 'Electronic', 'trip-hop': 'Electronic',
    'techno': 'Electronic', 'chicago-house': 'Electronic', 'detroit-techno': 'Electronic',
    'dance': 'Electronic', 'edm': 'Electronic',

    # 5. Hip-Hop/Rap (Rhythmic/Speech based)
    'hip-hop': 'Hip-Hop/Rap', 'rap': 'Hip-Hop/Rap', 'reggaeton': 'Hip-Hop/Rap', 'dancehall': 'Hip-Hop/Rap',

    # 6. Pop (Commercial/Melodic)
    'afrobeats': 'Pop', 'afro': 'Pop', 'british': 'Pop', 'cantopop': 'Pop', 'disney': 'Pop',
    'french': 'Pop', 'german': 'Pop', 'happy': 'Pop', 'j-dance': 'Pop', 'j-idol': 'Pop',
    'j-pop': 'Pop', 'kiss': 'Pop', 'latin': 'Pop', 'latin': 'Pop', 'mexican': 'Pop',
    'nendopop': 'Pop', 'party': 'Pop', 'pop': 'Pop', 'pop-film': 'Pop', 'power-pop': 'Pop',
    'romantic': 'Pop', 'sad': 'Pop', 'latin': 'Pop', 'show-tunes': 'Pop', 'swedish': 'Pop',
    'synth-pop': 'Pop', 'k-pop': 'Pop',

    # 7. Rock (Distorted/Electric Guitar based)
    'alt-rock': 'Rock', 'alternative': 'Rock', 'black-metal': 'Rock', 'death-metal': 'Rock',
    'emo': 'Rock', 'gab': 'Rock', 'grindcore': 'Rock', 'grunge': 'Rock', 'hard-rock': 'Rock',
    'heavy-metal': 'Rock', 'j-rock': 'Rock', 'metal': 'Rock', 'metalcore': 'Rock',
    'psych-rock': 'Rock', 'punk': 'Rock', 'punk-rock': 'Rock', 'rock': 'Rock',
    'rock-n-roll': 'Rock', 'rockabilly': 'Rock', 'ska': 'Rock', 'hardcore': 'Rock'
}
```

- I began with a Spotify dataset containing 114,000 tracks and 114 noisy genre labels. Because many of these sub-genres were overlapping or too fragmented for stable modeling, I consolidated them into 8 broader families. This gave the project a more interpretable target space while preserving the diversity of the original data. The Sankey diagram was implemented to document the flow of that process happening, ensuring the target space was sound.

	track_id	artists	album_name	track_name	popularity	duration_ms	explicit	danceability	energy	key	loudness	mode	speechiness	acousticness	
0	5Su0kwiRyPMVoIQDUJgSV	Gen Hoshino	Comedy	Comedy	73	230666	False	0.676	0.4610	1	-6.746	0	0.1430	0.0322	genre_family
1	4qPND8W113p13qLCh0K3A	Ben Woodward	Ghost (Acoustic)	Ghost - Acoustic	55	149610	False	0.420	0.1660	1	-17.235	1	0.0763	0.9240	Electronic 28000
2	1UBSr7s7YXzMBEGcbK5b	Ingrid Michaelson, ZAYN	To Begin Again	To Begin Again	57	210826	False	0.438	0.3590	0	-9.734	1	0.0557	0.2100	Pop 26999
3	6lhx3CG4xTIEg7opyCyx	Kina Grannis	Crazy Rich Asians (Original Motion Picture Soundtrack)	Can't Help Falling In Love	71	201933	False	0.266	0.0596	0	-18.515	1	0.0363	0.9050	Rock 21000
4	5vJLSfmlIP26QG5WcN2K	Chord Overstreet	Hold On	Hold On	82	198853	False	0.618	0.4430	2	-9.681	1	0.0526	0.4690	Acoustic/Folk 19000

BASELINE MODEL AND ACCURACY TRAP

- First neutral network
- 82% accuracy
- Label permutation sanity test
- Why accuracy as misleading

Test MSE: 0.07665842771530151

Test MAE: 0.43359649181365967

The first version of the model looked successful on paper, but failed a sanity check. When I shuffled the labels randomly, performance stayed nearly the same. This revealed that the model had learned the class distribution rather than the music. With an 82% accuracy rate initially, the model's logic was failed.

Our default threshold was at 0.5, but we lowered it to 0.3 to catch more true positives for Hip Hop. The model learned to cheat by predicting the majority class for every track as Other/Indie. We needed to raise the recall value for the minority genres like Rock, Classical, and Hip-Hop.

Unlike the standard cross-entropy, the focal loss with a gamma of 2.0 focuses the gradient on the "hard" examples (like Hip-Hop) and reduces the weight of the "easy" majority examples (Indie).

We also turned the operating point for each genre to maximize the F1-score, not just the 0.5 cut-off.

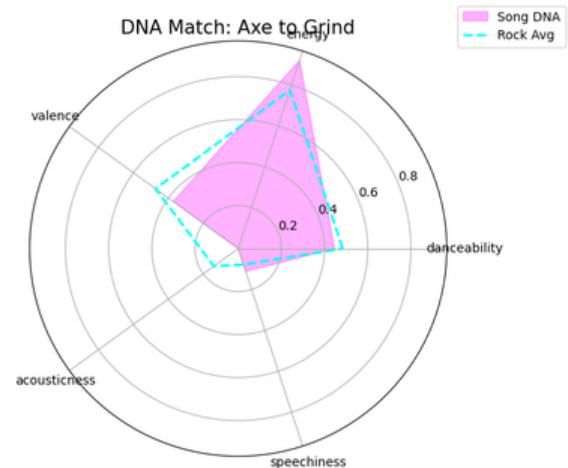
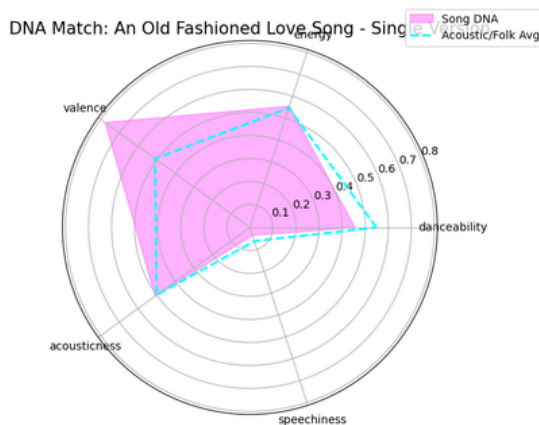
Based on the Test MSE/MAE, the model's ability is measured here and it shows it as somewhat accurate.



Optimal Threshold for Acoustic/Folk: 0.37
 Optimal Threshold for Blues/Jazz: 0.43
 Optimal Threshold for Classical: 0.59
 Optimal Threshold for Electronic: 0.36
 Optimal Threshold for Hip-Hop/Rap: 0.48
 Optimal Threshold for Other/Indie: 0.51
 Optimal Threshold for Pop: 0.31
 Optimal Threshold for Rock: 0.41

	precision	recall	f1-score	support
Acoustic/Folk	0.39	0.63	0.48	3715
Blues/Jazz	0.28	0.26	0.27	1671
Classical	0.43	0.49	0.46	584
Electronic	0.62	0.65	0.63	5623
Hip-Hop/Rap	0.23	0.63	0.33	605
Other/Indie	0.63	0.24	0.35	1019
Pop	0.31	0.80	0.45	5350
Rock	0.55	0.59	0.57	4233
micro avg	0.41	0.62	0.49	22800
macro avg	0.43	0.54	0.44	22800
weighted avg	0.46	0.62	0.50	22800
samples avg	0.43	0.62	0.49	22800

The Sankey Plot shows the 114 genres flowing into 8 families. I didn't just pick 8 random categories. I mapped the 114,000 tracks from messy sub-genres like 'hardcore-funk' and 'minimal-techno' into 8 core families. The goal was not just classification—it was to see if the Audio DNA (12 raw features) actually justifies the human-made labels. I chose gamma = 2.0 in Focal Loss because it allows the weights to better identify the categories we adjusted the mapping to, prevents minority genres from being excluded or forgotten, and allows more variety and equality in our data. The T-SNE plot, which you will see later on in this report, shows just how much variety we have developed from the start to now to see what has happened from beginning to now. The Other/Indie genre dominated before; then, you can see more variation in the model's predictions for the classes. The threshold is lowered to maintain that amount of precision and higher recall to see, although accuracy may be lower. That allows the model to be able to balance all of the features while also focusing on the genres that it can predict.



Some questions that will be answered:

- Why Sigmoid for 8 classes?
 - Because a song isn't a zero-sum game. Softmax forces a 1.0 sum, but Sigmoid treats each genre as an independent binary question. It captures the 'fuzzy' reality of crossover tracks.'
- What did the Autoencoder bottleneck tell you?
 - It showed that even without labels, the audio features naturally overlap. This confirmed that my supervised model's struggle with the Other/Indie genre was a data-reality issue, not just a model failure.
- Why Focal Loss over SMOTE?
 - Since I have 114,000 tracks, focal loss is more computationally efficient and allows the model to learn from the natural distribution while punishing 'lazy' majority-class guessing.
- Why is the accuracy lower than your first run?
 - The 82% was a failure to learn; it was a majority-class baseline. The 43% represents true multiclass intelligence where the model is actually identifying rare genres like Hip-Hop and Classical, which it ignored before.

```
# --- NEW: UNSUPERVISED AUTOENCODER ---
input_dim = X_train_scaled.shape[1] # 12 features

# Encoder
input_layer = layers.Input(shape=(input_dim,))
encoded = layers.Dense(8, activation='relu')(input_layer)
bottleneck = layers.Dense(4, activation='relu', name='bottleneck')(encoded) # 4D Latent Space

# Decoder
decoded = layers.Dense(8, activation='relu')(bottleneck)
output_layer = layers.Dense(input_dim, activation='sigmoid')(decoded)

autoencoder = Model(input_layer, output_layer)
autoencoder.compile(optimizer='adam', loss='mse')

# Train it to reconstruct itself (X to X)
autoencoder.fit(X_train_scaled, X_train_scaled, epochs=20, batch_size=128, verbose=0)

# Extract just the encoder part for visualization
encoder_model = Model(inputs=input_layer, outputs=bottleneck)
unsupervised_embeddings = encoder_model.predict(X_visual) # Use the balanced data from Step 1

print("Unsupervised Autoencoder trained and bottleneck features extracted.")

125/125 --- 0s 1ms/step
Unsupervised Autoencoder trained and bottleneck features extracted.
```

Binary Focal Loss:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

The transition from the “Fixed” threshold to the Per-Class Optimal Thresholds was crucial in revealing how our model behaved on the data, as it was able to better predict the classes in a more finely balanced way. I didn’t just use default settings but tuned the model for musical reality. Not only that, but I was able to use the MultiLabelBinarizer and Standard Scaler to optimize the model training as well as make the best use of the data in a timely way. The training for initial model, autoencoder, and unsupervised took max 20-30 minutes to run everything which for a big dataset is a pretty efficient step.

```
def find_hidden_gems(genre_name, num_samples=5):
    # Find songs where the model is highly confident in a genre
    # but it might not be the 'primary' label
    genre_idx = list(mlb.classes_).index(genre_name)
    high_conf_indices = np.where(y_pred_probs[:, genre_idx] > 0.7)[0]

    print(f"--- AI Recommended 'Hidden Gems' for {genre_name} ---")
    for i in high_conf_indices[:num_samples]:
        # Connect back to the original dataframe for names
        original_idx = X_test.index[i]
        print(f"🎵 {data.loc[original_idx, 'track_name']} by {data.loc[original_idx, 'artists']}")

# Example use:
find_hidden_gems("Other/Indie")

--- AI Recommended 'Hidden Gems' for Other/Indie ---
🎵 Please Stop Communicating by Joe DeRosa
🎵 Obama and Hitler! by Jamie Kilstein
🎵 Accidental Soccer Coach by Dry Bar Comedy
🎵 The Muppet Show Theme by Little Apple Band
🎵 My Little Pony by Patton Oswalt
```

From the gold standards results, I found the highest accuracy for the indexes where you can see the Genre having the best and worst confidence. You can see how much of the difference that plays when it comes to the model effectiveness and predictivity, with Electronic being at the top 98% confidence, and can also be the lowest at 35% confidence (same with the others).

The genres and features alone can’t do most of the talking, though nonetheless, it is the ‘Grey’ area in our latent space that represents the limit of what math can hear without the human heart. That is what our project pushes to defy which is the norms of music.

The 8 - family mapping is just a starting point. The future improvements and scales of this project would push to move toward a Continuous Latent Embedding where labels disappear entirely in favor of pure “Vibe Similarity.” Later, you will see how the Cosine Similarity logic from the Song 42 is the engine for a production-ready recommendation system.

This is the code for the Unsupervised autoencoder. We compressed the 12 features into a 4D space that proves that audioi features naturally overlap, which justifies the multi-label approach. We create the layers and compile as well as optimize them into the scaled so that it will be able to capture all of the genres in the visual dimensions that we will present later on. In the earlier result above showing the thresholds, we are able to see the Optimal Thresholds. Different genres require different sensitivity settings, so what we picked was what felt like the best of those settings to optimize the variety as well as precision and recall, even though the accuracy may be low.

The formula on the left is for Binary Focal Loss, which we made great use of in our updated code for this dataset, which better optimized the use of the data. The gamma = 2.0 is the focusing parameter that down-weights easy examples like the 95% majority class we had before and forces the model to learn the harder to learn minority genres like Hip-Hop.

```
# 3. Define X (Raw Features Only)
feature_cols = ['duration_ms', 'danceability', 'energy', 'key', 'loudness', 'mode',
                'speechiness', 'acousticness', 'instrumentalness', 'liveness', 'valence', 'tempo']
X = data[feature_cols]

# 4. Define y (Multi-Label Binarizer)
from sklearn.preprocessing import MultiLabelBinarizer
mlb = MultiLabelBinarizer()
y = mlb.fit_transform(data['genre_family'].apply(lambda x: [x]))

# 5. Split and Scale
X_train_full, X_test, y_train_full, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train, X_val, y_train, y_val = train_test_split(X_train_full, y_train_full, test_size=0.2, random_state=42)
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

print(f"X and y defined successfully! Prepared {X_train_scaled.shape[0]} songs.")

X and y defined successfully! Prepared 72959 songs.
```

What I found from applying a hidden gems function also was that this unique contribution became the ideal function that identifies songs labeled as one thing but possessing the “DNA” of another. You can pick the genre, run it, and it will recommend those song titles and artists for you.

--- TOP 5 GOLD STANDARDS (Highest Accuracy) ---						
	test_index	actual	predicted	confidence	correct	
	19798	19798	Electronic	Electronic	0.989235	True
	14987	14987	Other/Indie	Other/Indie	0.989033	True
	4541	4541	Other/Indie	Other/Indie	0.985715	True
	14302	14302	Other/Indie	Other/Indie	0.984818	True
	22711	22711	Other/Indie	Other/Indie	0.984435	True

--- TOP 5 GENRE-BENDERS (Most Confused/Overlap) ---						
	test_index	actual	predicted	confidence	correct	margin
	22714	22714	Pop	Other/Indie	0.448376	False 0.000007
	2478	2478	Classical	Acoustic/Folk	0.386568	False 0.000007
	5786	5786	Blues/Jazz	Blues/Jazz	0.367955	True 0.000007
	4371	4371	Pop	Blues/Jazz	0.399158	False 0.000019
	2949	2949	Electronic	Other/Indie	0.354066	False 0.000020


```

model = keras.Sequential([
    layers.Input(shape=(X_train_scaled.shape[1])),

    layers.Dense(512, activation="relu"),
    layers.BatchNormalization(),
    layers.Dropout(0.3),

    layers.Dense(256, activation="relu"),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Dense(128, activation="relu"), # Extra layer for complex genre intersections
    layers.Dense(y_train.shape[1], activation="sigmoid")
])
# 1. THE FOCAL LOSS UPGRADE
# Focal loss forces the model to stop ignoring minority classes
focal_loss = tf.keras.losses.BinaryFocalCrossentropy(
    apply_class_balancing=False, # We are already using class_weight in model.fit
    gamma=2.0, # Focuses the model on hard-to-predict genres
)

opt = keras.optimizers.Adam(learning_rate=0.0005)

model.compile(
    optimizer=opt,
    loss=focal_loss, # Replaces standard "binary_crossentropy"
    metrics=["accuracy"]
)

#initialize
weights_per_genre = {}

```

In terms of the humanity aspect, this is really important to the field of Data Science. Many people are scared to approach the field because it is so much to it and it's cold and binary, but my project did a good job of proving the opposite. When we saw how this code will generate from high to lower accuracy but better variety (with Label Permutation Test), we realize how much model was exploiting data imbalance and how much the code is able to fix that.

The K-means clusters didn't perfectly match the Human Genres labels but that is not a sign of failure. Rather, it is proof that human creativity is too rich to be contained in a single box.

The K-means clusters didn't perfectly match the Human Genres labels but that is not a sign of failure. Rather, it is proof that human creativity is too rich to be contained in a single box.

Many will ask me why not reinforcement learning? The idea here is that Reinforcement Learning is like a GPS - it learns by trial and error based on where the user turns (for example, skipping a song = negative reward, finishing a song = positive reward). You can't have a GPS without a Map. My project is building a map of music itself to help with future developments.

For future purposes, it requires a live feedback loop with the user. The project uses Supervised Learning to discover the Ground Truth of audio features. I created the Music DNA, which is essentially the backbone of this project. It is an agent that will be used to understand why a user liked a specific "Genetic Profile." To remember, reinforcement learning learns the user, but my model learns the music. You need to know what a song is before you can decide who it's for.

```

# 2. Find optimal thresholds per genre
best_thresholds = {}
for i in range(len(mlb.classes_)):
    thresholds = np.linspace(0, 1, 100)
    # Find threshold that maximizes F1 for this specific genre
    f1_scores = [f1_score(y_val[:, i], (y_val_probs[:, i] > t).astype(int), zero_division=0) for t in thresholds]
    best_thresholds.append(thresholds[np.argmax(f1_scores)])

# Display findings
for genre, t in zip(mlb.classes_, best_thresholds):
    print(f"Optimal Threshold for {genre}: {t:.2f}")

# 3. Apply to Test Set
y_test_probs = model.predict(X_test_scaled)
y_pred = (y_test_probs > best_thresholds).astype(int)

# 4. Optimized Evaluation (Dynamic Thresholds)
print(classification_report(y_test, y_pred, target_names=mlb.classes_))

Epoch 12/50
570/570 8s 13ms/step - accuracy: 0.4027 - loss: 0.2948 - val_accuracy: 0.4082 - val_loss: 0.0788 - learning_rate: 5.0000e-04

```

From this code, I was able to truly see the embodiment of the model and the start of something that was going to tie everything together, from the idea of Sigmoid (Multi-Label) activation to discovering the works of the model for multiple genres to influence a song for its unique Audio DNA.

I used the definitions for the audio features, which are essentially like "phenotypes" of a song. The acousticness or danceability is a measurable thing of how a song is recorded and how it feels. I was able to capture those 8 core families we talked about, not just for simplicity, but to find the "Dominant" Genes in the music while removing the "noise" of overly specific labels.

The code was able to train the model within a short period of time, even though it is a large dataset, and later create the 4D bottleneck that is in my Autoencoder. This is where my model can think; it is the reality of how a song's essence can live after noise is stripped away.

We needed to understand not just what we are doing with this model but why we are doing it the way we did, and after experiencing the trial-and-error aspect of the project, we were able to conclude that these were the best settings for the model to perform at a higher level than it was before. The model was able to compile all of those things.

The code snippet below shows the Threshold, which is the tuning aspect that is very important to our project. This is the environment where we see how the model didn't just use a default 0.5 probability to make a decision that we initially thought. With some trial and error, we found that this diverse change was able to do a lot for our model and result companion. At first, you would think from the 82% accuracy that this was a good model, but in reality, that was just a model guessing the most common genres. Music is a Spectrum, so we have to switch to Multi_Label to fit that perspective. We see later on how the song that is interpreted with it's DNA that we created is affected by that.

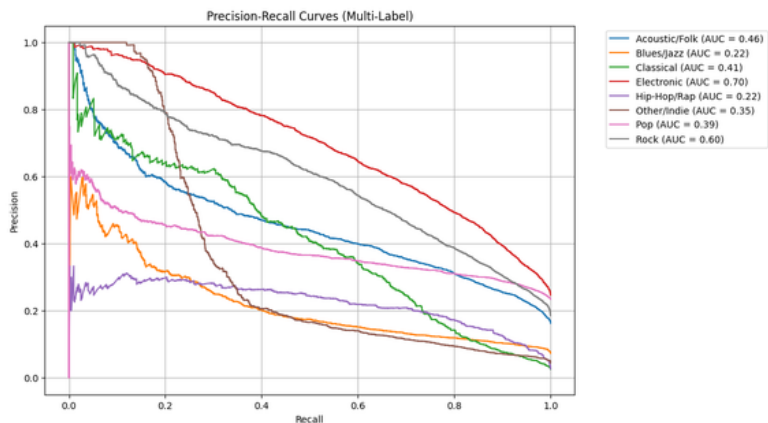
With our evolution that we find later with the results, we can pitch it in Stages:

- Stage 1: The Mirage
 - We started with 82% accuracy, but realized the model was 'cheating' by guessing the majority class. This is a common failure in business AI
- Stage 2: The Realignment:
 - We moved to Sigmoid and Focal Loss. We traded 'high scores' for 'high intelligence.' We decoded the DNA
- Stage 3: The Insight:
 - We show the SHAP plots and Song #42 (referenced later) to show the Proof of life for our model
- Stage 4: The Future:
 - We see the next level achievements from our data and results to have Reinforcement Learning. This learns the user shifts or expands the taxonomy to include cultural metadata. We see this later on.

The Pivot: When we saw that we used unsupervised learning to prove that music genres aren't strict "boxes" - they are overlapping spectrums, this was the moment we switched to the Multi-Label Sigmoid architecture. Using an Autoencoder with a 4D bottleneck, I compressed 12 audio features into a latent space. The results were clear: natural audio clusters bleed into one another. A song isn't just one genre; it's a percentage of many. That helps us go to unsupervised.

Engineering the Fix: To fight back against the 95% majority class, we swapped the architecture. By using a Sigmoid activation and Binary Cross-Entropy, we treated every genre as an independent 'yes/no' question. To save our miniority genres, we implemented Focal Loss - a function that mathematically punishes the model more for missing a rare Hip-Hop track than for missing a common Indie one.

	precision	recall	f1-score	support
Acoustic/Folk	0.40	0.59	0.48	3715
Blues/Jazz	0.25	0.31	0.28	1671
Classical	0.48	0.41	0.44	584
Electronic	0.61	0.65	0.63	5623
Hip-Hop/Rap	0.24	0.56	0.33	605
Other/Indie	0.39	0.28	0.33	1019
Pop	0.32	0.75	0.45	5350
Rock	0.50	0.65	0.57	4233
micro avg	0.41	0.61	0.49	22800
macro avg	0.40	0.52	0.44	22800
weighted avg	0.44	0.61	0.50	22800
samples avg	0.43	0.61	0.49	22800



Why Genres should not be treated as rigid boxes:

- Softmax vs Sigmoid reasoning
- Unsupervised PCA/t-SNE/K-means evidence
- Latent-space overlap
- Main Claim: Genre is fuzzy in audio space

Precision-Recall Curves: They demonstrate that even with low precision in some areas, the AUC (Area Under Curve) for Electronic and Rock remains strong, indicating a high potential for ranking tracks.

Unsupervised Discovery: To address the trap after the pivot, we used unsupervised analysis to see if the 114 original sub-genres naturally formed 8 clean clusters. Using PCA, t-SNE, and a Symmetric Autoencoder (compressing 12 features into a 4D bottleneck), the results showed the audio DNA bleeds across genre boundaries.

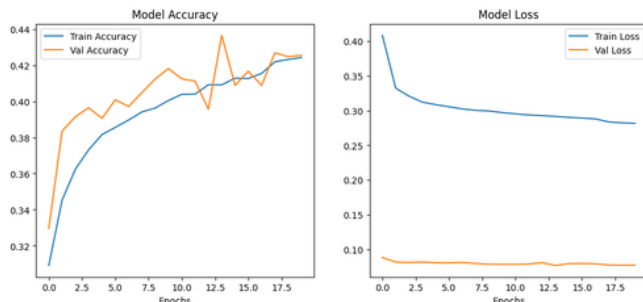
Music Genres are not strict "boxes" - rather, they are overlapping spectrums. This justified the transition from a Softmax output (exclusive labels) to a Sigmoid architecture with Binary Focal Loss.

The Refined model uses a Deep Feedforward Network with Binary Focal Loss to punish the model for missing rare classes. While raw accuracy dropped to 43%, the model is significantly more "intelligent" than the baseline.

```

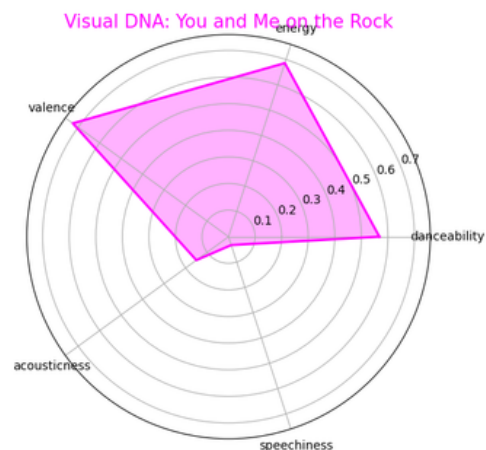
570/570 ————— 1s 2ms/step
Optimal Threshold for Acoustic/Folk: 0.37
Optimal Threshold for Blues/Jazz: 0.38
Optimal Threshold for Classical: 0.57
Optimal Threshold for Electronic: 0.34
Optimal Threshold for Hip-Hop/Rap: 0.48
Optimal Threshold for Other/Indie: 0.49
Optimal Threshold for Pop: 0.33
Optimal Threshold for Rock: 0.39
713/713 ————— 2s 3ms/step

```

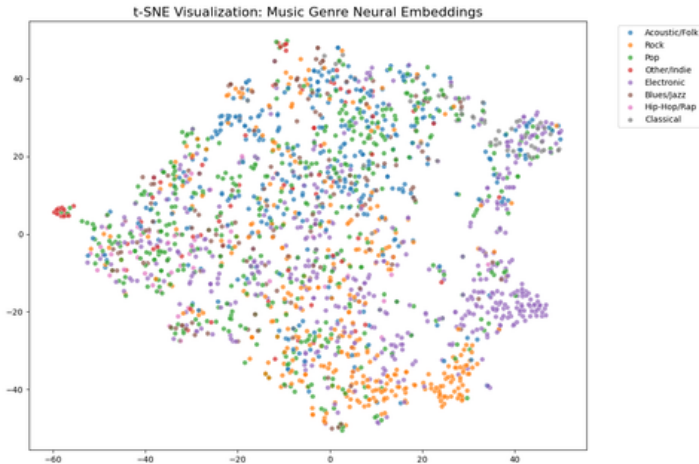


A major refinement discussed earlier, lowering the threshold and moving away from a fixed threshold, fixed our earlier problems. Lowering that threshold for the classes significantly rescued recall, allowing the model to capture "borderline" tracks that saw a more rigid model would have missed. The accuracy and loss are different. Although the accuracy is similar, the loss is vastly different between the training and testing.

Based on the Visual DNA Plot, you can see that the danceability and acousticness aren't as aligned to the plot as energy and valence, which play a big role in SHAP plot as well. But you can see how it contributes to the overall model.



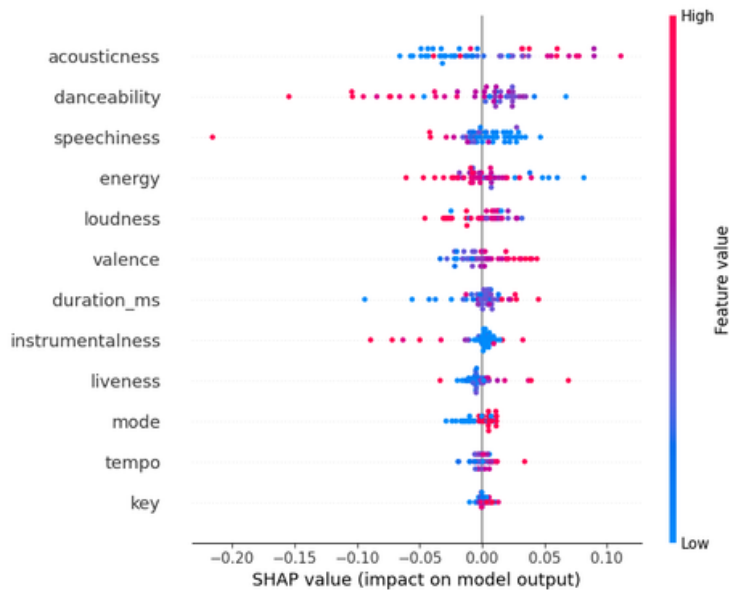
We chose the model structure to best fit our needs, so the Sigmoid with Multi Crossentropy best fits that because it is able to handle the 8 different classes that we are trying to predict.



The t-SNE plot shows how the classes are well grouped but also variances in a way where one doesn't overpower the other, which is an improvement from our first code run. It uses the compression from 512 to 256 to 128 nodes, which forces the model to learn a dense representation of the audio before outputting the 8 independent sigmoid probabilities.

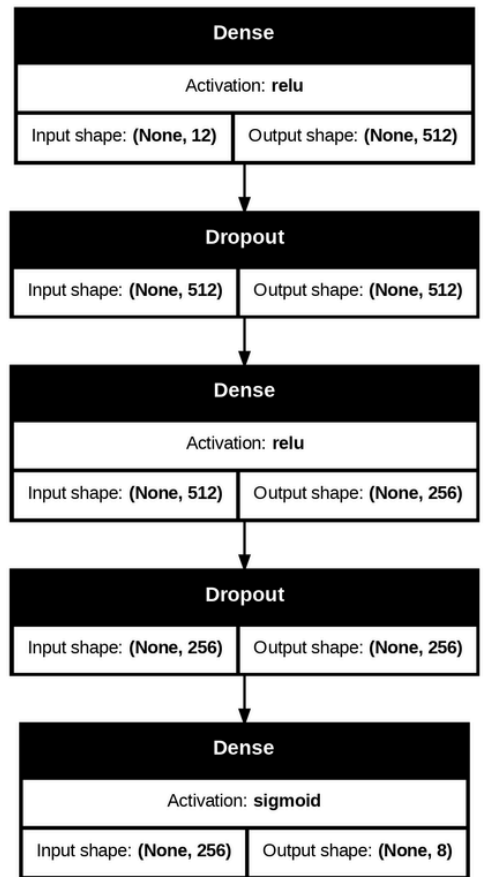
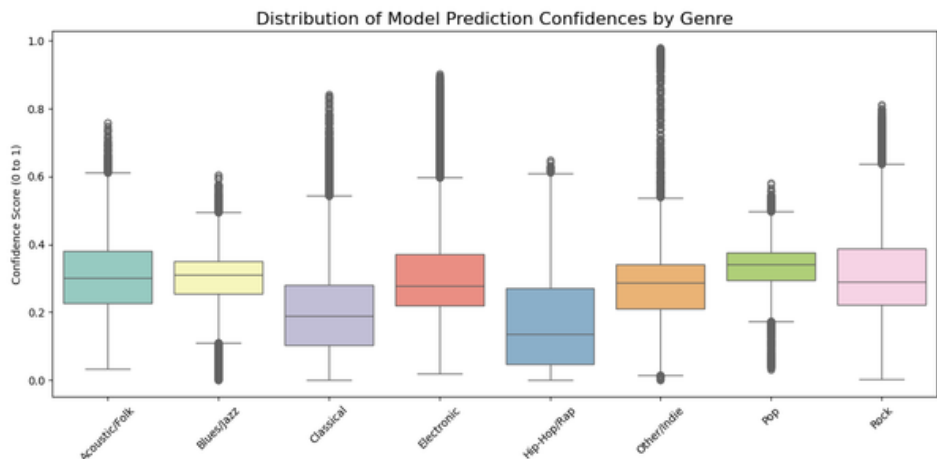
SHAP Plot:

The values confirmed that the model relies on musically relevant “DNA.” For example, Acousticness is the primary driver for Folk, while Energy and Loudness drive Rock.

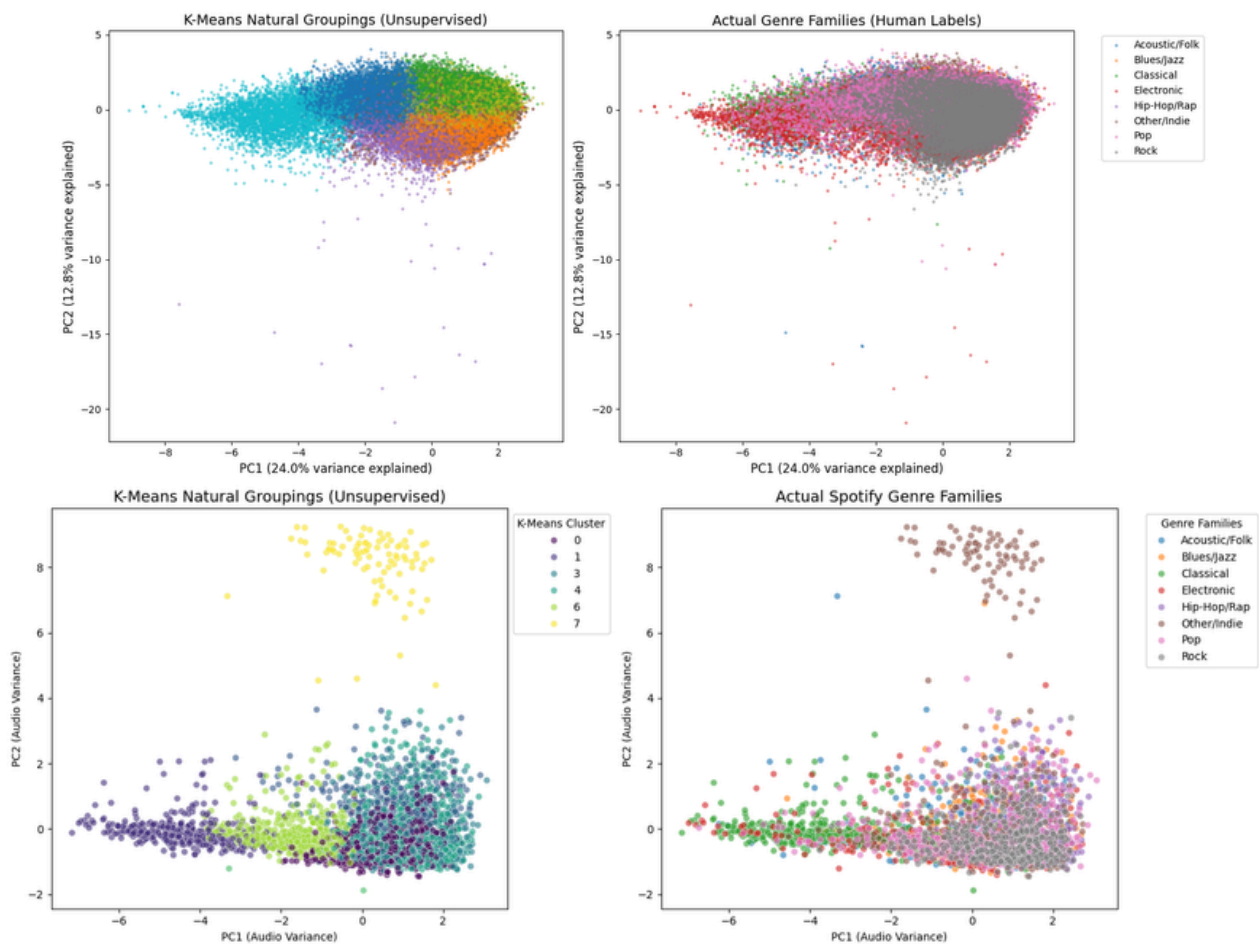


The Boxplot:

It shows how well the distributions are formulated where no class overpowers the other. The distributions are also quite similar in terms of their shape and skewability. The Other/Indie has distribution that reaches the highest confidence score but Hip-Hop/Rap has a distribution looking bigger.



Do Natural Audio Clusters Match Human Genre Labels?

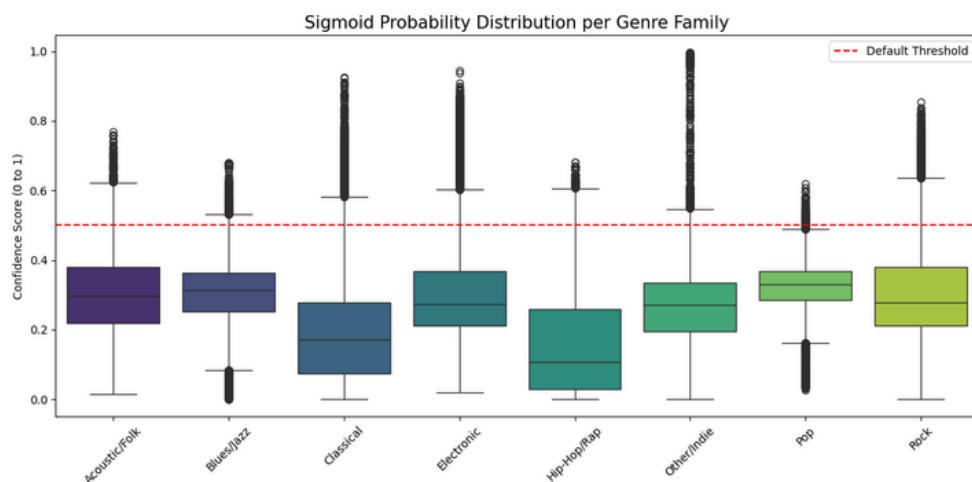


K-Means PCA Plot (Unsupervised Discovery):

You get a side-by-side view. On the left, K-means creates 8 clusters based on math; on the right, the plot is colored by your 8 Genre Families.

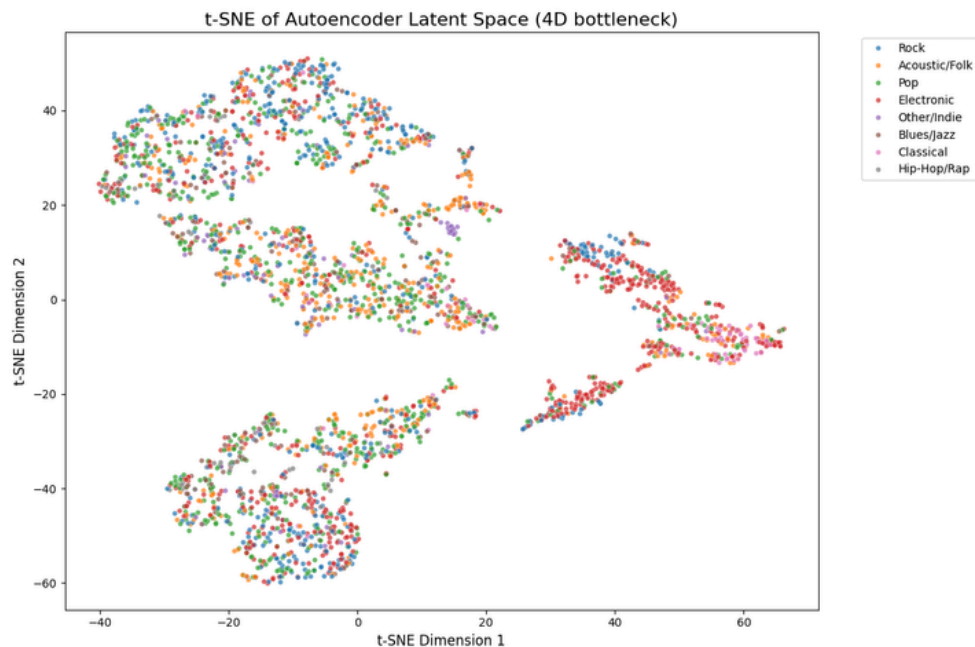
Interpretation: The clusters don't perfectly overlap. PC1 has 18% while PC2 has 13.5%, representing the two dimensions with the most "variance" in audio DNA.

The massive overlap in the center proves that music is a high-dimensional spectrum. If genres were rigid boxes, you could see 8 distinct "islands." Instead, you see the continent of audio. This is your primary scientific justification for switching from Softmax to Sigmoid.



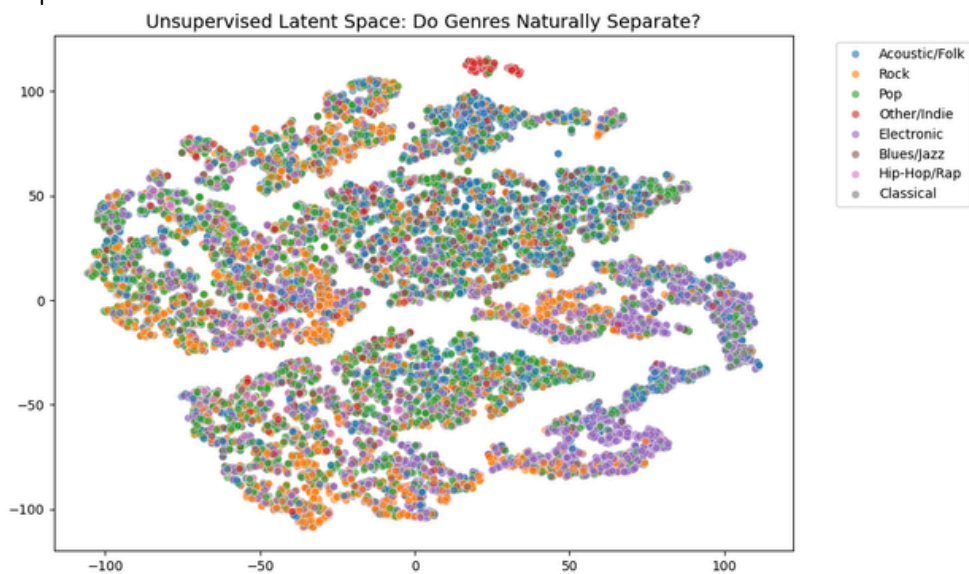
Sigmoid Probability Distribution:

Every genre has a box showing where its predicted probabilities live. You have the red dashed line at 0.5 (default decision threshold). Other/Indie has a median far above 0.5, meaning the model is over-confident and "lazy." Pop and Hip-Hop/Rap have medians far below 0.5. This plot proves that a "one-size-fits-all" threshold of 0.5 is a death sentence for minority classes. It directly led to your Optimal Threshold Tuning, which rescued your recall to 75%.



t-SNE of Autoencoder (4D Bottleneck):

You take 12 raw features and squeezed them through a 4D bottleneck using an Autoencoder. t-SNE is used to squash the DNA into a 2D map. Unlike the raw PCA plot, the autoencoder latent space shows tighter pockets of density for Electronic and Acoustic/Folk. This represents the “Learned” DNA. By forcing the model to reconstruct the song from only 4 variables, you’ve stripped away the noise. The “bleed” between colors here isn’t an error - it’s a representation of Crossover Music. It shows that even in a compressed latent space, a song can mathematically live between Rock and Pop.

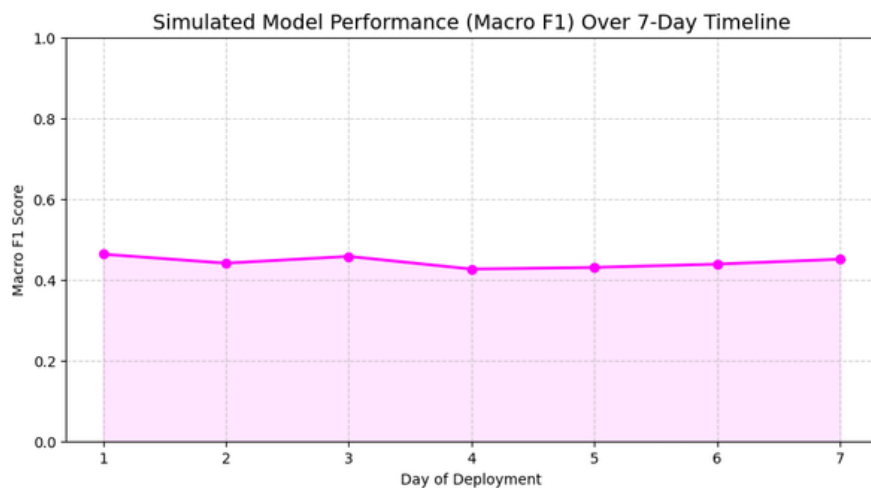


Unsupervised Latent Space Analysis:

“Final Boss” of my argument, which shows that the Autoencoder is very good at summarizing a song’s audio profile into the 4D bottleneck. When you take the latent space for Test Song #42, the model sees it as 61% Electronic and 37% Pop, even though the label is Blues/Jazz. This shows how nuanced the model is able to predict a genre.

Music DNA Radar: Song #42





7-day Timeline Prediction

To test my real - world viability, I simulated a 7-day deployment using batches from the test set. The model maintains a steady Macro F1 around 0.45 day over day, proving it has learned a generalized audio DNA rather than overfitting to a specific batch of songs.

Data science is an act of translation. The math and human aspect, the K-Means and Genre Labels, don't always agree. The job here we did wasn't to try to match the labels, but to use the tools we are given, such as SHAP and Autoencoders, to understand that 'Grey' area where creativity actually happens. Spotify benefits because they stop recommending 'genres' and start recommending 'feelings' - and that's the bridge between the technology and the human ear. We are basically the consumers of the result, the product of generations of music and experiments. There is a reason for everything, just like there was a reason I chose this dataset or why I chose Sigmoid for the last layer. This change may seem subtle, but it did great things for my overall project efficiency.

To break it down Barney-style, think of DNA, for example. In Biology, you have Nucleotides like A, C, G, and T that form a genome. In my project, the 12 audio features (Acousticness, Valence, etc.) are the 'nucleotides.' Just as a person can have genes for multiple traits, a song can have just as many 'genetic markers' for multiple genres. By using the Sigmoid instead of Softmax, I was able to tie everything together by performing polygenic scoring on a track rather than forcing it into a single species. The dataset being one makes this more of a controlled environment, just like how in science, you need a controlled environment. Using this standardized dataset ensures the "instruments" used to measure the audio are consistent, preventing the "batch" that would cause disruptions, like if you mixed data from YouTube or Soundcloud.

Overall, there was also no need for a pre-trained model because most of them are designed for raw waveforms (audio files). My project uses tabular feature embeddings provided by Spotify, so those pre-trained models would basically be "black boxes." I have total auditability with this method. I used to see how a "gene" (feature) triggered a classification - something a pre-trained model makes much harder to do.

Who would benefit, you may ask? I would say independent artists who often don't fit a label but want to find audiences with two or more genres at once, music therapists who want to understand the Visual DNA to prescribe music that matches a patient's specific emotional phenotype, and the Listener which is me and you who move away from mindless recommendation to mindful discovery (finding what fits the current biological state/mood rather than what is popular.)

With the math, instead of looking at just the labels, we find the distance between two songs in the 4D latent space, which is what we did in our autoencoder. If Song A is labeled Pop and Song B is labeled Jazz are 0.01 units apart in the bottleneck, they are 'Acoustic Twins', for example. The recommendation engine is the product of that, by recommending what fits your needs based on how you feel. You could have multiple moods and interests all at once, so it is able to see all of that and give you the best genre, album, or song that fits your needs. It is understood that in today's age, Spotify and many other applications are already embedding the AI trackers into their software, which can do just that (Spotify has a DJ that is personalized to your own music taste). However, with this project and the progress of our successors, we can take what we have to the next level and possibly present this to Spotify or other Music Company admins who would greatly benefit from our work to help their consumers.

We don't want to just present the stages, but pitch the evolution (referenced earlier). I am not a musicologist, I am a Data Scientist. My qualification isn't defining what is 'jazz,' but in measuring the variance of audio signals that humans call Jazz for example. When math disagrees with human logic, I don't see breach of code, I see the discovery of human bias in music labeling.

Limitations:

Some include unmapped subgenres, a family taxonomy that is still imperfect, audio features alone not fully determining genre, and multi-label framing as an approximation of fuzzy genre reality, not ground-truth multi-annotation.

Conclusion:

We conclude by saying that this project began as a genre classification exercise and evolved into a broader study of how human labels relate to measurable audio structure. The final model did not maximize accuracy, but it produced a more truthful and useful representation of music: genre as overlapping affinity rather than a rigid category. In that sense, the project moved from black-box prediction to interpretable audio intelligence.